S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

Practical No. 03

Aim:- Analyze suitable technique to Build a Tic-Tac-Toe Game Engine with an AI Player in Python.

Name of Student:

Roll No.:

Semester/Year: VI/III

Academic Session: 2025-2026

Date of Performance:

Date of Submission:

AIM: Analyze suitable technique to Build a Tic-Tac-Toe Game Engine with an AI Player in Python.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To be able to understand the concept of Tic-Tac-Toe.
- To be able to acquire the puzzle solving capability.
- To be able to acquire the game development ability.

THEORY:

Tic-tac-toe is a simple and popular two-player game where the objective is to get three of your symbols in a row on a 3x3 grid. The game is typically played with Xs and Os. Here's how it works:

How to Play:

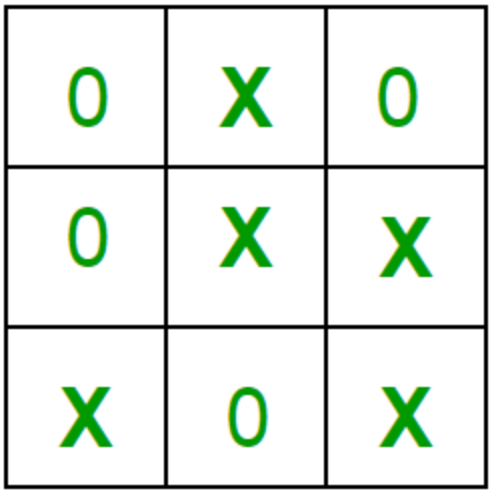
1. **Players:** Two players take turns, one playing as "X" and the other as "O."
2. **Grid:** The game board consists of a 3x3 grid.
3. **Objective:** The first player to place three of their marks (either Xs or Os) in a horizontal, vertical, or diagonal row wins the game.
4. **Turns:** Players take turns placing their mark in an empty square on the grid.
5. **Winning:** The game ends when one player gets three of their marks in a row or when all squares are filled, resulting in a draw if no one has won.

Strategies:

- **First move advantage:** Going first can be an advantage if played strategically.
- **Blocking:** Always watch your opponent's moves and block them if they are about to get three in a row.
- **Center square:** The center square is often the best first move, as it provides more opportunities to win.

Rules of the Game

- The game is to be played between two people (in this program between HUMAN and COMPUTER).
- One of the player chooses 'O' and the other 'X' to mark their respective cells.
- The game starts with one of the players and the game ends when one of the players has one whole row/ column/ diagonal filled with his/her respective character ('O' or 'X').
- If no one wins, then the game is said to be draw



Implementation In our program the moves taken by the computer and the human are chosen randomly. We use rand() function for this. **What more can be done in the program?** The program is in not played optimally by both sides because the moves are chosen randomly. The program can be easily modified so that both players play optimally (which will fall under the category of Artificial Intelligence). Also the program can be modified such that the user himself gives the input (using scanf() or cin). The above changes are left as an exercise to the readers. **Winning Strategy – An Interesting Fact** If both the players play optimally then it is destined that you will never lose ("although the match can still be drawn"). It doesn't matter whether you play first or second.In another ways – " Two expert players will always draw ". Isn't this interesting ?

PROGRAM CODE:

ALGORITHM:

- **Initialize the Game Board:**
  - Create a 3x3 grid (a 2D array or list of lists) filled with empty values.
- **Display the Game Board:**
  - Create a function to print the current state of the grid to the console.
- **Set the Players:**
  - Assign one player as "X" and the other as "O."
  - Decide randomly or allow one player to choose who goes first.
- **Main Game Loop:**
  - Loop until the game ends (either a player wins or the grid is full):
    1. **Display the Board:**

- Print the current state of the grid.

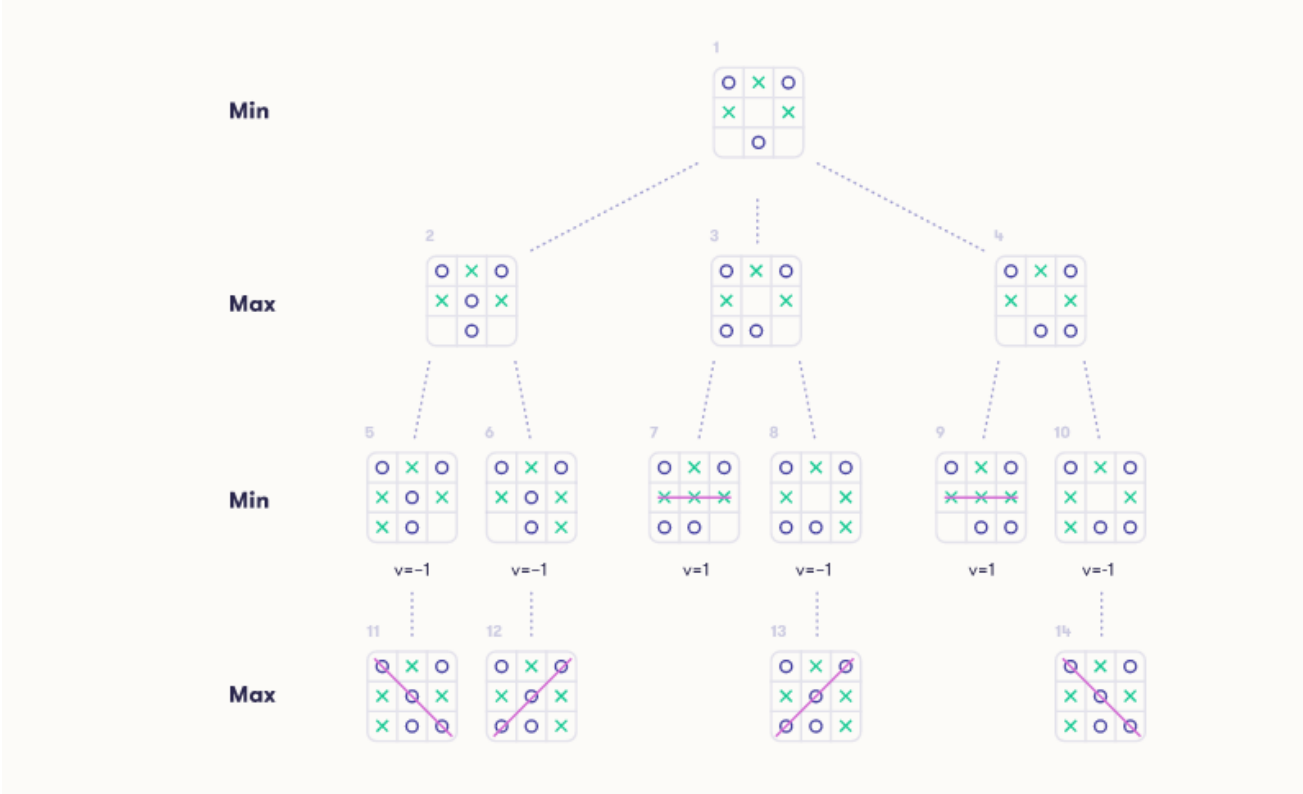
2. **Player Input:**
   - Prompt the current player to select a row and column (1-3) for their move.
   - Validate the input to ensure the chosen cell is empty and within bounds.
3. **Update the Board:**
   - Place the current player's symbol (X or O) in the selected cell.
4. **Check for a Win:**
   - Create a function to check if the current player has won:
     - Check all rows, columns, and both diagonals to see if any have three of the same symbols.
5. **Check for a Draw:**
   - If the grid is full and no one has won, declare a draw.
6. **Switch Players:**
   - Alternate between players (X -> O, O -> X).

- **End the Game:**

  - If a player wins, announce the winner.
  - If the game ends in a draw, announce that the game is tied.

Search Space:

Consider nodes (5)–(10) on the second level from the bottom. In nodes (7) and (9), the game is over, and Max wins with three X's in a row. The value of these positions is +1. In the remaining nodes, (5), (6), (8), and (10), the game is also practically over, since Min only needs to place her O in the only remaining cell to win. In other words, we know how the game will end at each node on the second level from the bottom. We can therefore decide that the value of nodes (5), (6), (8), and (10) is also –1.



CODE :-

```
import math
import time

board = [" " for _ in range(9)]

def print_board():
    print("\n")
    print(f"{board[0]} | {board[1]} | {board[2]}")
    print("--+---+--")
    print(f"{board[3]} | {board[4]} | {board[5]}")
    print("--+---+--")
    print(f"{board[6]} | {board[7]} | {board[8]}")
    print("\n")

def check_winner(player):
    win_conditions = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for condition in win_conditions:
        if board[condition[0]] == board[condition[1]] == board[condition[2]] == player:
            return True
    return False

def is_draw():
    return " " not in board

def minimax(is_maximizing):
    if check_winner("O"):
        return 1
    if check_winner("X"):
        return -1
    if is_draw():
        return 0

    if is_maximizing:
        best_score = -math.inf
        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                score = minimax(False)
                board[i] = " "
                best_score = max(score, best_score)
        return best_score
    else:
        best_score = math.inf
```

```
        for i in range(9):
            if board[i] == " ":
                board[i] = "X"
                score = minimax(True)
                board[i] = " "
                best_score = min(score, best_score)
        return best_score

def computer_move():
    best_score = -math.inf
    move = 0
    for i in range(9):
        if board[i] == " ":
            board[i] = "O"
            score = minimax(False)
            board[i] = " "
            if score > best_score:
                best_score = score
                move = i

    print("Computer is thinking...")
    time.sleep(1)

    board[move] = "O"
    print(f"Computer chose position: {move + 1}")

def play_game():
    print("You are X | Computer is O")
    print("Positions are numbered 1 to 9 like this:\n")
    print("1 | 2 | 3")
    print("--+---+--")
    print("4 | 5 | 6")
    print("--+---+--")
    print("7 | 8 | 9\n")

    while True:
        print_board()

        try:
            player_move = int(input("Choose position (1-9): ")) - 1
            if player_move not in range(9) or board[player_move] != " ":
                print("Invalid move. Try again.")
                continue
        except:
            print("Please enter a number between 1 and 9.")
            continue

        board[player_move] = "X"
```

```
        if check_winner("X"):
            print_board()
            print("You Win!")
            break

        if is_draw():
            print_board()
            print("It's a Draw!")
            break

        computer_move()

        if check_winner("O"):
            print_board()
            print("Computer Wins!")
            break

        if is_draw():
            print_board()
            print("It's a Draw!")
            break

play_game()
```

INPUT & OUTPUT:

```
You are X | Computer is O
Positions are numbered 1 to 9 like this:

1 | 2 | 3
--+---+--
4 | 5 | 6
--+---+--
7 | 8 | 9



  |   |
--+---+--
  |   |
--+---+--
  |   |


Choose position (1-9): 5
Computer is thinking...
Computer chose position: 1


O |   |
--+---+--
  | X |
--+---+--
  |   |


Choose position (1-9): 3
Computer is thinking...
Computer chose position: 7
```

```
O |   | X
--+---+--
  | X |
--+---+--
O |   |


Choose position (1-9): 6
Computer is thinking...
Computer chose position: 4


O |   | X
--+---+--
O | X | X
--+---+--
O |   |


Computer Wins!
```

CONCLUSION:

DISCUSSION QUESTIONS:

- Which Technique is best for above game?

- Give the best rules to win the game.

- Give the best rules to draw the game.

REFERENCES:

- https://course.elementsofai.com/2/3
- https://www.geeksforgeeks.org/implementation-of-tic-tac-toe-game/
- https://www.geeksforgeeks.org/low-level-design-of-tic-tac-toe-system-design/
- https://react.dev/learn/tutorial-tic-tac-toe